

Diagnostic et définition du x5

Le produit actuel suit deux parcours distincts. À l'ouverture, `terrain_bootstrap.py:235-329` construit ou relit le heightmap terrestre CPU 1024 × 512, puis `terrain_conditioning.py:338-438` en dérive les cinq entrées physiques du réseau ; le panorama distant peut présenter ces entrées avant que le coarse soit appris. À proximité, `index.html:85-110` publie la vue et demande successivement LOD 4, 3, 2, 1, 0 ; `terrain_lod.js:36-65` bloque l'étape suivante jusqu'au dessin soumis. Le serveur choisit ensuite `preview`, `coarse`, `latent` ou `decoder` dans `terrain_server.py:382-450`. Le x5 doit donc nommer un résultat : temps entre action de caméra et couverture native LOD 0 visible, à seed nouvelle et poids résidents, puis séparément démarrage sans poids résidents, premier aperçu mondial, tuile isolée et revisite disque. Accélérer un forward seul ne démontre aucun de ces résultats.

Mesures archivées, à ne pas présenter comme nouveau profilage :

`docs/RUNTIME_LOD_OPTIMIZATION.md` rapporte pour le profil naturel et des stores NN neufs une médiane LOD 3 de 2,117 s, LOD 2 apparié de 3,829 s, avec forte variabilité LOD 2 et bureau GPU actif. Le LOD 3 emploie 169 fenêtres initiales et 121 finales, 19 appels base, ainsi que quatre fenêtres coarse et 80 appels coarse. LOD 2 emploie 202 à 244 fenêtres base et 25 fenêtres decoder. Les revisites du store vivant, 12 à 28 ms, ne sont pas du terrain nouveau. Le reçu terrestre

`E:/TerrainDiffusionRuntime/terrestrial-bootstrap-runtime/browser-final-cold.json` observe 12,443 s jusqu'à la couverture LOD 0, 22 misses, premier LOD 4 à 4,570 s et LOD 2 à 10,518 s ; la revisite du même lieu termine en 0,570 s avec 22 hits. Ce sont deux sessions, pas une distribution de latence. Plusieurs SHA du reçu diffèrent des fichiers actuels, dont `index.html`, `terrain_renderer.js`, `terrain_inference.py` et `terrain_server.py` : ce parcours historique n'est pas une baseline des sources présentes. La charge externe est inconnue. Le reçu CPU de bootstrap, une autre seed, observe 1,182 s de création earthlike et 0,037 s de relecture vérifiée ; ne pas les additionner au navigateur.

Déduction : viser x5 sur 12,443 s imposerait au plus 2,489 s dans un protocole comparable. Selon Amdahl, si 20 % du trajet est incompressible, le plafond tend vers x5 quand le reste devient infiniment rapide ; si 10 % reste fixe, la fraction de 90 % doit gagner x9. Il faut mesurer les parts réelles avant tout engagement. `terrain_jobs.py:155-204` sérialise le calcul, mais décale l'écriture/encodage sur deux workers CPU. Quatre transferts navigateur dans `index.html:103` ne donnent donc pas quatre calculs simultanés. Les temps de préparation, file, NN, fusion, transport et upload doivent être tracés sur la même requête sans sommer des intervalles imbriqués. Aucun profil Macro A4 retiré ne quantifie ce bootstrap.

Moteur des trois réseaux : priorité aux poids et aux formes réelles

Le checkpoint local épinglé décrit trois EDMUnet2D, non un réseau unique. Le coarse a 11 canaux d'entrée, six de sortie, 128 canaux internes, un seul niveau, deux blocs par niveau, aucune attention et une entrée 64^2 au runtime ; `terrain_inference.py:142-198` répète ses convolutions vingt fois par fenêtre avec solveur. Le base reçoit cinq canaux plus un vecteur conditionnel de 58 valeurs ; ses quatre niveaux ont 192, 384, 576 et 768 canaux, trois blocs par niveau et attention dans le midblock (8×8 au runtime), les niveaux d'attention étant choisis d'après `image_size` configuré. Le decoder a quatre niveaux de 64, 128, 192, 256 canaux, trois blocs par niveau, sans attention, et reçoit des fenêtres 512^2 . Configurations : les trois `config.json` du snapshot local `xandergos/terrain-diffusion-30m` ; assemblage dans `terrain-diffusion/terrain_diffusion/models/edm_unet.py:109-183`, UNetBlock dans `unet_block.py:81-153`. Les coûts plausibles sont convolutions BF16, normalisations MP, activation, interpolation, copies de skip, puis QK/softmax/V du base. Il faut compter FLOP, lectures mémoire et lancements par forme, pas supposer que l'attention domine.

Le runtime a déjà supprimé deux overheads évidents. `terrain_inference.py:88-115` met en cache les poids MP normalisés ; `terrain_nn_constants.py:25-95` préconstruit coefficients, filtres et échelles, en préservant leur arrondi BF16. L'attention réellement patchée dans `terrain_nn_constants.py:84-96` utilise `einsum`, softmax et `einsum` ; la branche SDPA du fichier upstream est inaccessible dans ce chemin. `terrain_inference.py:260-311` compacte les patches de bruit en une copie H2D par batch réel. `terrain_cuda_graphs.py:47-117` capture seulement des formes admises, avec validation exacte et buffers privés ; le profil ordinaire garde les graphs coarse et base jusqu'à batch 4, tandis que base 16 et decoder restent eager (`terrain_inference.py:31-77`, 414-450). Répéter ces optimisations comme gains neufs serait un double comptage.

Hypothèse principale : spécialiser les forwards aux poids figés, dimensions et dtypes effectivement employés, en prédisposant poids et activations et en fusionnant les successions MPConv-somme/concaténation-activation qui créent des tenseurs intermédiaires. Un prototype limité au base batch 16 puis au decoder 512^2 peut comparer une implémentation dédiée à PyTorch eager, `torch.compile/Inductor` et TensorRT, chacun dans un processus isolé. Le chemin `torch.compile` est désactivé sur Windows par `world_pipeline.py:338-345` et `terrain_server.py:181-182` ; l'évaluer sur Linux demande un budget de compilation et une preuve d'identité, et ne promet rien sur le produit Windows. TensorRT exigerait import, profils de formes, plugins éventuels pour opérations MP et maintenance de version ; ONNX n'est actuellement ni le backend serveur ni un raccourci validé. `webgpu/` est un probe distinct du calcul PyTorch CUDA BF16.

Le batch est un état numérique, pas un simple curseur de débit : `terrain_inference.py:405-410` rejette coarse et decoder supérieurs à un, et le scheduler garde les frontières de batch `InfiniteTensor` (`terrain_window_scheduler.py:60-89`). Regrouper des fenêtres pour augmenter l'occupation peut modifier l'arrondi, l'ordre de fusion et le terrain. Les changements de layout NHWC, graph base 16 et kernels ciblés sont des expériences séparées, validées sur sortie finale et pic VRAM ; aucun gain n'est acquis sans profil par opérateur.

Génération, maths, caches et affichage

Le bootstrap CPU produit le raster natif, remappe séparément terres et profondeurs via quantiles pondérés, vérifie SHA et persiste NPZ (terrain_bootstrap.py:161-221, 235-306). Une optimisation de tri, I/O ou validation ne touche le NN que via sa latence d'ouverture ; le reçu d'une seed earthlike donne déjà environ 1,18 s froid, donc un x5 global ne peut reposer sur ce seul bloc. La fabrique de conditionnement échantillonne hauteur, latitude et climat, avec tables de quantiles et rasters fixes (terrain_conditioning.py:161-222, 338-438). Cibles à chronométrer : lecture initiale des rasters, création des statistiques, allocations par ligne, interpolation répétée aux mêmes coordonnées. Cacher un résultat intermédiaire exige une clé contenant seed u64, style, paramètres des cinq canaux, sources, binaire et poids ; une clé trop large invalide inutilement, trop étroite mélange des mondes.

Sur la génération apprise, terrain_server.py:283-312 échantillonne les champs coarse/latent et interpole avec scipy.map_coordinates ; les grandes vues sont découpées en blocs source de 48. terrain_server.py:314-373 calcule les moyennes d'aire coarse par chunks 32^2 et retransfère chaque rectangle CUDA vers CPU avant de former les blocs NumPy. C'est un candidat précis au profilage des synchronisations et de la redondance des fenêtres, surtout aux LOD 9 à 11 ; conserver exactement alignement global, extension de bord et moyenne en mètres. LOD 2 à 0 emploie world.get puis une réduction CUDA avant copie de la tuile (terrain_server.py:382-411). Le climat appris reste coarse, échantillonné sur une grille 33^2 , et le filtrage gaussien du relief $\text{LOD} \geq 3$ s'exécute ensuite sur CPU (terrain_server.py:415-450 ; terrain_climate.py:10-53). Fusionner les prélèvements de même footprint peut éviter des lectures et copies, sous réserve de l'ordre arithmétique.

Les caches travaillent à plusieurs niveaux : fenêtres RAM, fenêtres coarse persistées avec poids et contrôle de somme (terrain_coarse.py:279-431), heightmaps, mips DEM final, tuiles physiques et réponses PNG. terrain_final_mips.py:20-48 lit les enfants exacts ; terrain_server.py:758-827 déduplique une tuile, puis écrit height et climat ; terrain_server.py:832-900 relit les NPY pour transporter un Float32 binaire ou produire un PNG. Une représentation persistée ou mise en mémoire des octets de réponse pourrait réduire les hits HTTP, mais le reçu chaud à 0,570 s sert de borne pratique : optimiser un hit ne réduit pas la fabrication d'une nouvelle région. Le cache coarse mondial d'environ 707 Mo et 6 160 fenêtres appartient à une préparation explicite ; l'augmenter, ou précharger hors champ, déplace le coût et l'espace disque plutôt que de rendre le calcul gratuit.

Le navigateur impose une barrière de dessin à chaque niveau et quatre téléchargements (index.html:61-68, 85-110). WebGPU reçoit par tuile 304^2 Float32 de hauteur plus 5×33^2 Float32 de climat, soit 391 444 octets bruts ; terrain_renderer.js:170-242 crée des textures R32, exécute deux flous puis ombrage et compose. Le cache client a un budget 192 Mio (index.html:25, 47). Mesurer allocations, uploads, draw soumis et présentation réelle avant modification. Une fusion de tuiles voisines ou un transport compressé changerait CPU, bande passante et latence dans des sens opposés : gain hypothétique à tester.

Protocole, décisions et coût

Première décision : figer un scénario terrestre représentatif. Pour trois seeds dont 0, 42 et une u64, inclure côte, intérieur, latitude élevée, coordonnées négatives et pan continu. Mesurer séparément création du monde, premier aperçu, LOD 4/3/2 et couverture LOD 0, avec poids froids puis résidents, fenêtres NN neuves, disque chaud et revisite HTTP. Conserver viewport, backend WebGPU, préparation, autres clients et préchargement. Les scripts `benchmark_terrain_bootstrap.py`, `benchmark_runtime_lod.py`, `benchmark_runtime_progression.py` et `benchmark_runtime_navigation.cjs` donnent des protocoles complémentaires. Tracer requêtes et fenêtres entre `index.html:99-110`, `terrain_jobs.py:122-215` et `terrain_server.py:758-900`. Rapporter séparément CUDA events et wall clock, sans additionner les intervalles imbriqués. Aucun calcul GPU n'a été lancé pour ce rapport.

Deuxième décision : établir un Pareto instrumenté. Si le base batch 16 domine, comparer eager, fusion ciblée, layout, compilation et moteur spécialisé sur les mêmes fenêtres. Si le decoder domine LOD 2 ou le 30 m, travailler sa forme 512² et les 25 fenêtres sans changer stride 384 ni fusion. Si le coarse domine le premier affichage, examiner ses vingt étapes en gardant le solveur intact. Si le temps est surtout file/attente, revoir admission, dépendances et priorité du visible. Si bootstrap, climat, NPY, transport ou upload dominant, optimiser ces postes. Chaque essai a un témoin, un seul changement et un reçu de source.

Critère go/no-go x5 : pour une base mesurée et gelée, médiane du trajet froid terrestre $\leq 20\%$ de la base et maximum/p95 empirique non dégradés sur un corpus assez grand pour les interpréter ; premier aperçu et pan continu ne doivent pas régresser. Rapporter également p50, p95, mémoire GPU réservée/allouée, disque par monde, compilation, temps de chargement et coût de maintenance. Rejeter toute revendication fondée seulement sur un forward accéléré ou sur un hit. Amdahl doit être recalculé après chaque essai avec les parts résiduelles : les gains d'un graph, du batching et d'une fusion qui suppriment les mêmes lancements ne se multiplient pas.

Fidélité : comparer hauteur et cinq climats en mètres et unités d'origine, données finales et fenêtres intermédiaires, y compris zéros signés, NaN et bords de tuiles. Exiger d'abord égalité octet par octet sur les chemins prétendus exacts, avec référence CPU des arrays et SHA des sources importées ; en cas de divergence backend, appliquer la barrière ≤ 1 m max et les limites climatiques existantes, puis inspection côtière et mosaïque. Préserver seed, ordre du RNG portable, batches, BF16, checkpoints, solveur et identité des caches. Les expériences approximatives - pas réduits, FP16, quantification, interpolation nouvelle - forment une branche distincte avec acceptation explicite de leur coût qualité. Un moteur sur mesure représente des semaines de développement et un risque de maintenance ; si son gain total mesuré ne franchit pas la trajectoire x5 après prototype limité, arrêter et privilégier les améliorations ciblées. Aucun gain cité ici n'est une mesure nouvelle.